Machine Learning Assignment Report

Submission report · generated from experiment outputs · reproducibility seed: 42

Assignment overview

This report evaluates classification, regression, clustering, and dimensionality reduction. Experiments use fixed public datasets, and numerical results below are read from saved run outputs. The classification and regression parts use an 80/20 train/test split. Clustering and PCA are unsupervised full-dataset demonstrations and do not use a test split.

1. Classification

Dataset and setup

The classification experiment uses the UCI AI4I 2020 Predictive Maintenance dataset ([UCI dataset page](https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset)), 10,000 observations. The binary target is `Machine failure`; inputs are product type and five sensor measurements. `UDI` and `Product ID` are identifiers; failure-mode flags are excluded to avoid target leakage. A fixed seed 42 stratified 80/20 split is shared by KNN, decision tree, RBF-SVM, and CNN. The three classical models use stratified five-fold training-only CV, selecting positive-class F1. Preprocessing is inside their pipelines. The CNN takes the eight preprocessed tabular inputs in a fixed `(8, 1)` sequence and uses Conv1D layers; training uses only the training split with a validation subset for checkpointing. This sequence interpretation is a limitation because adjacent columns are not naturally spatial.

Held-out metrics

Precision, recall, and F1 treat failure as the positive class. ROC-AUC uses probabilities or continuous decision scores. The best ROC-AUC is **RBF SVM (0.9767)**.

Classifier	accuracy	precision	recall	f1	roc_auc
RBF SVM	0.9800	0.7593	0.6029	0.6721	0.9767
1D CNN	0.9340	0.3261	0.8824	0.4762	0.9714
Decision Tree	0.9860	0.8226	0.7500	0.7846	0.8910
KNN	0.9600	0.4032	0.3676	0.3846	0.6742

Confusion counts (rows actual, columns predicted):

Classifier	TN	FP	FN	TP
KNN	1895	37	43	25
Decision Tree	1921	11	17	51
RBF SVM	1919	13	27	41
1D CNN	1808	124	8	60

ROC curves:

ROC-AUC assesses ranking across thresholds, while F1 and confusion counts use a particular threshold. The data are imbalanced, so accuracy alone understates the challenge. Model complexity differs: KNN can overfit at very small neighbor counts, trees can overfit as leaves/depth increase, RBF-SVM controls boundary flexibility through C and gamma, and CNN capacity/epoch selection affects generalization. Consult saved CV tables and CNN learning history for train-validation gaps.

2. Regression

Dataset and setup

The required regression experiment uses the bundled scikit-learn diabetes dataset (442 records, ten numeric baseline measurements, continuous one-year disease progression target; [scikit-learn dataset documentation](https://scikit-learn.org/stable/datasets/toy_dataset.html#diabetes-dataset)). The fixed seed 42 split has 353 training and 89 test records. Gradient-descent linear regression explicitly minimizes `J(w,b)=mean((Xw+b-y)^2)/2`; its batch gradients are `X.T @ residual / n` and `mean(residual)`, with standardized inputs, learning rate 0.1, maximum 20,000 iterations, and a loss-change tolerance/patience stopping

 rule. Polynomial regression applies degree-two feature expansion and linear regression. Ridge and Lasso alpha values are selected by five-fold CV on training data only, inside standardized pipelines.

Held-out results

Pearson **R** is shown separately from coefficient of determination **R²**.

Model	mse	rmse	pearson_r	r2
Gradient descent linear	2900.1916	53.8534	0.6780	0.4526
Polynomial degree 2	3096.0283	55.6420	0.6743	0.4156
Ridge	2892.0146	53.7775	0.6786	0.4541
Lasso	2884.6243	53.7087	0.6795	0.4555

Polynomial degree comparison, using training fit and training-only five-fold validation. Degree zero is a mean-only baseline and demonstrates underfitting through high training and validation error; degree five nearly interpolates training rows but has much larger validation error, showing overfitting:

degree	train_mse	cv_mse	cv_mse_std
0	6076.3980	6136.6552	870.9716
1	2868.5497	3074.1421	265.1860
2	2393.1386	3926.3144	554.5356
3	749.5951	2443780.7107	2424351.2399
5	0.0000	81498.3988	28853.3002

Gradient descent stopped after 10570 iterations. The convergence check was `True`; loss changed from 3038.199006 to 1434.274851.

Regularization CV results (lowest validation MSE selected):

Ridge: selected alpha **1**.

alpha	cv_mse	cv_mse_std
0.0010	3074.1351	265.1893
0.0100	3074.0729	265.2188
0.1000	3073.5213	265.4828
1.0000	3071.9314	266.4168
10.0000	3078.5972	263.9944
100.0000	3144.2881	320.1253

Lasso: selected alpha **0.1**.

alpha	cv_mse	cv_mse_std
0.0010	3074.0626	265.2157
0.0100	3073.3938	265.4713
0.1000	3069.6107	264.1205
1.0000	3073.2878	258.8704
10.0000	3341.8342	400.4612
100.0000	6136.6552	870.9716

Increasing polynomial degree generally lowers training error while validation error reveals the point where added flexibility stops generalizing. Ridge shrinks coefficients smoothly; Lasso can shrink coefficients to zero. Alpha is chosen without using the test set. Actual and predicted values are plotted in `outputs/assignment_regression/actual_vs_predicted.png`.

3A. K-Means from scratch

The required implementation is NumPy Lloyd K-Means with seeded k-means++ initialization, squared Euclidean assignment, centroid means, a shift tolerance, an iteration cap, and empty-cluster repair. It uses all four standardized numeric features of the bundled Iris dataset (150 records; [scikit-learn Iris documentation](https://scikit-learn.org/stable/datasets/toy_dataset.html#iris-plants-dataset)). Labels are not used for fitting. WCSS was evaluated for K=2, 3, 4, 5, and 6.

k	wcss	iterations	clusters_found
2	222.3617	3	2
3	139.8205	6	3
4	129.7774	5	4
5	104.0491	5	5
6	81.5814	9	6

WCSS cannot increase as K increases because extra centroids can only improve or preserve the closest-center fit. An elbow is subjective and can be weak or dataset-dependent; it is not proof of a uniquely correct K.

3B. PCA from scratch

PCA uses the ten numeric features of the bundled diabetes dataset. Features are standardized, centered, the sample covariance matrix is computed, then `numpy.linalg.eigh` yields eigenpairs sorted in descending order. Explained variance is eigenvalue divided by total eigenvalue; cumulative variance is its cumulative sum. Eigenvectors are checked for orthonormality, the covariance eigen-equation is checked numerically, and ratios sum to one.

Eigenvalues, ratios, and cumulative ratios:

component	eigenvalue	explained_variance_ratio	cumulative_explained_variance
1	4.0333	0.4024	0.4024
2	1.4957	0.1492	0.5517
3	1.2087	0.1206	0.6722
4	0.9576	0.0955	0.7678
5	0.6637	0.0662	0.8340
6	0.6041	0.0603	0.8943
7	0.5378	0.0537	0.9479
8	0.4347	0.0434	0.9913
9	0.0785	0.0078	0.9991
10	0.0086	0.0009	1.0000

Eigenvectors (columns are principal components):

Feature	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9	PC10
age	-0.2164	-0.0444	-0.4947	0.4140	-0.6869	0.2258	0.1095	-0.0149	-0.0081	-0.0033
sex	-0.1870	0.3865	0.1069	0.6799	0.3735	-0.0417	0.0676	-0.4429	0.0021	-0.0037
bmi	-0.3032	0.1563	-0.1675	-0.4998	0.1293	0.4032	0.5199	-0.3929	-0.0424	-0.0082
bp	-0.2717	0.1383	-0.5136	0.0197	0.4869	0.2728	-0.3207	0.4774	-0.0272	0.0032
s1	-0.3433	-0.5730	0.0686	0.0684	0.1292	-0.0054	-0.0736	-0.1294	0.0420	-0.7098
s2	-0.3519	-0.4559	0.2697	0.1678	0.1167	0.1333	0.2305	0.1913	0.3593	0.5632
s3	0.2824	-0.5062	-0.3860	0.0760	0.2450	-0.1064	0.0075	-0.3246	-0.4812	0.3174
s4	-0.4288	0.0682	0.3807	-0.0079	-0.1436	0.0339	-0.0712	0.1806	-0.7738	0.0906
s5	-0.3786	0.0262	-0.0636	-0.2644	-0.1516	-0.1787	-0.6473	-0.4497	0.1895	0.2645
s6	-0.3222	0.0849	-0.2768	-0.0871	0.0314	-0.8051	0.3573	0.1666	0.0153	-0.0026

The saved two-component output has shape [442, 2]; the three-component output has shape [442, 3].

Discussion and limitations

The experiments use one fixed split for supervised metrics. CV on training data guides hyperparameters; the held-out test is reserved for final comparison. Results may vary with a new sample or split. The CNN imposes an arbitrary sequence structure on tabular columns. Iris clustering uses standardized Euclidean distance and K-Means favors compact, roughly spherical clusters. PCA is linear and retains directions of variance rather than task-specific predictive information. Pearson R measures linear association, whereas R^2 measures explained variance relative to a mean baseline.

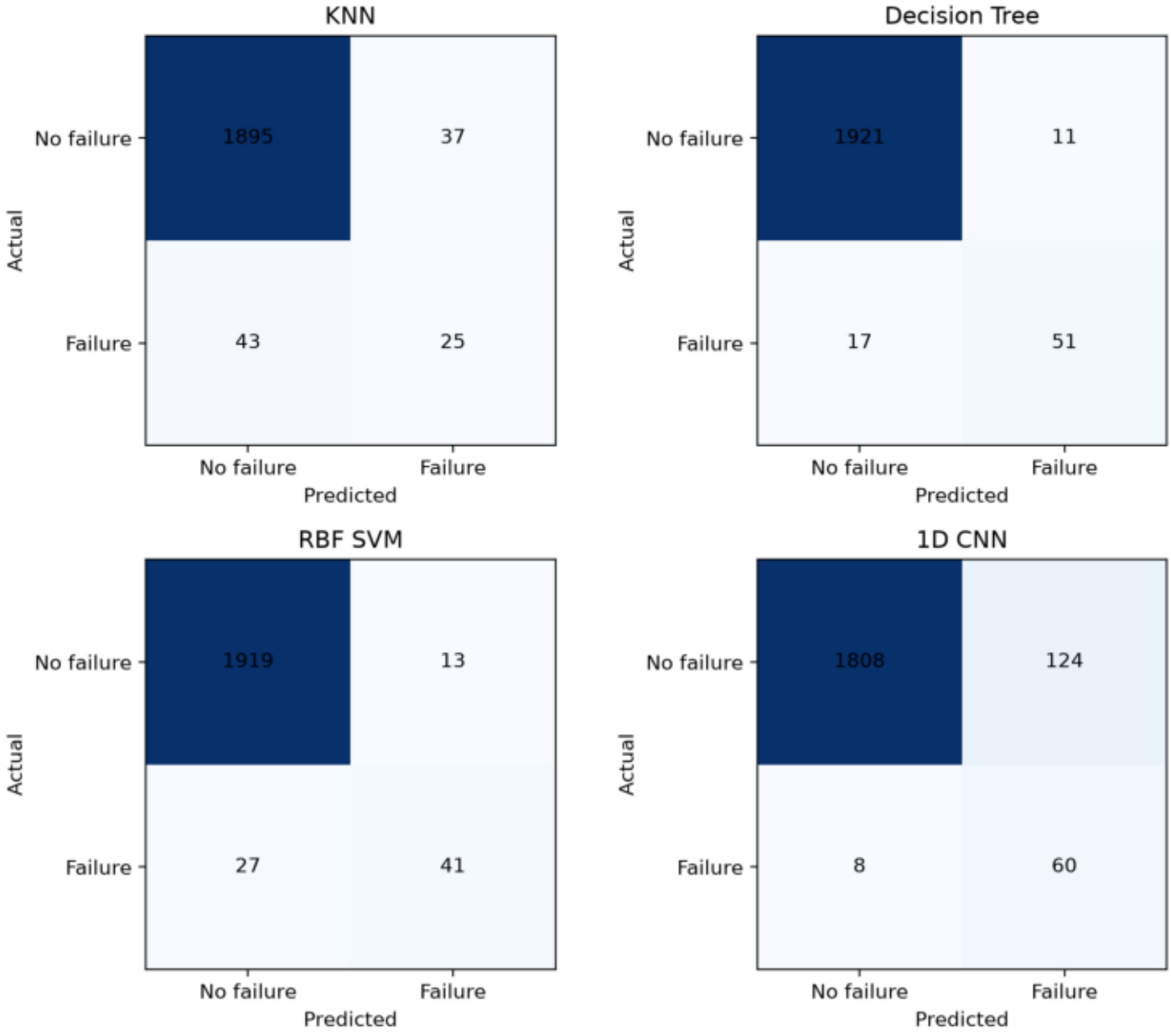
Reproduction

From the repository root, install `requirements.txt`, then run:

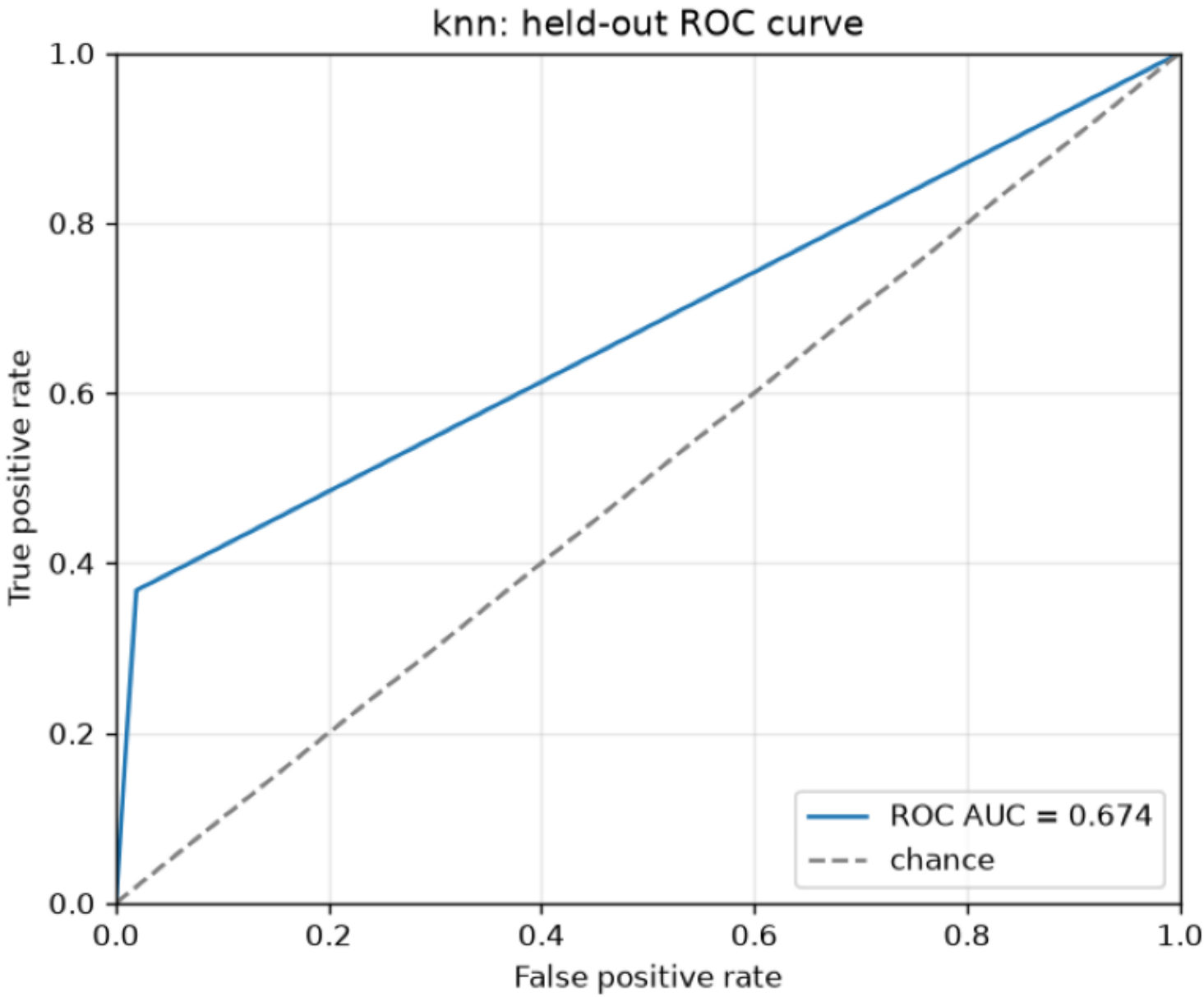
```
python -m classification.train
python -m cnn.train
python -m cnn.evaluate
python -m regression.assignment
python -m clustering.scratch_kmeans
python -m pca.scratch_pca
python -m reports.build_assignment_report
python -m unittest discover -v
```

Compliance checklist

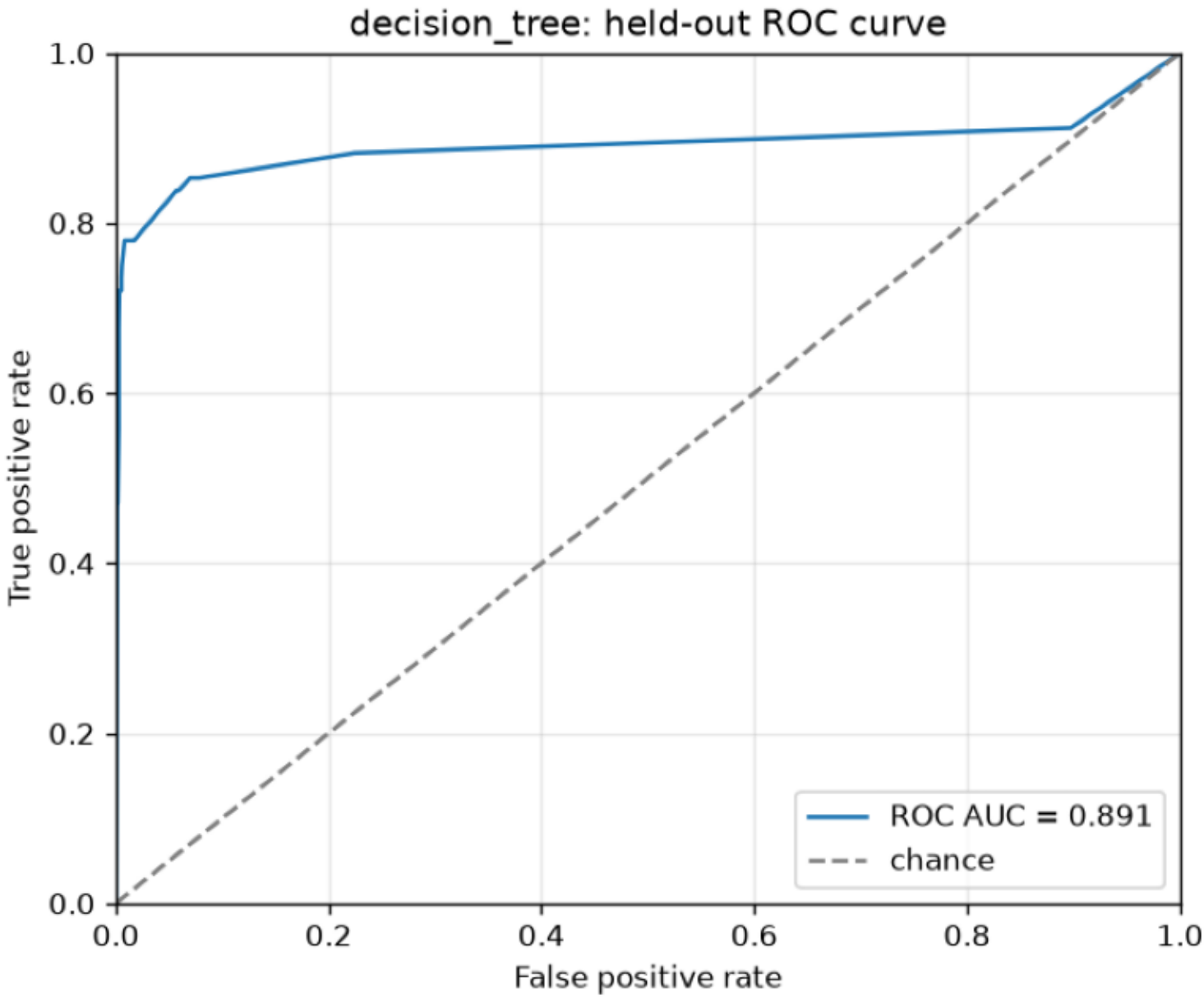
Requirement	Implementation	Generated evidence
KNN CV, tree, RBF-SVM, deep CNN; 80/20, seed, stratification; metrics, confusion matrices, ROC	`classification/train.py`, `cnn/`	`outputs/classification/`, `outputs/cnn/`
GD linear, polynomial, Ridge/Lasso; training-only CV, Pearson R, complexity curve	`regression/assignment.py`	`outputs/assignment_regression/`
NumPy K-Means, K=2–6, WCSS and elbow	`clustering/scratch_kmeans.py`	`outputs/assignment_clustering/`
Covariance/eigen PCA, two and three components, variance and visualization	`pca/scratch_pca.py`	`outputs/assignment_pca/`
Automated coverage for required from-scratch algorithms	`test_assignment_algorithms.py`	`python -m unittest discover -v`
Reproducible combined report	`reports/build_assignment_report.py`	`reports/ML_Assignment_Report.md`


classification_confusion_matrices.png
KNN
No failure
Failure
Actual
1895
37
43
25
No failure
Failure
Predicted
Decision Tree
No failure
Failure
Actual
1921
11
17
51
No failure
Failure
Predicted
RBF SVM
No failure
Failure
Actual
1919
13
27
41
No failure
Failure
Predicted
1D CNN
No failure
Failure
Actual
1808
124
8
60
No failure
Failure
Predicted

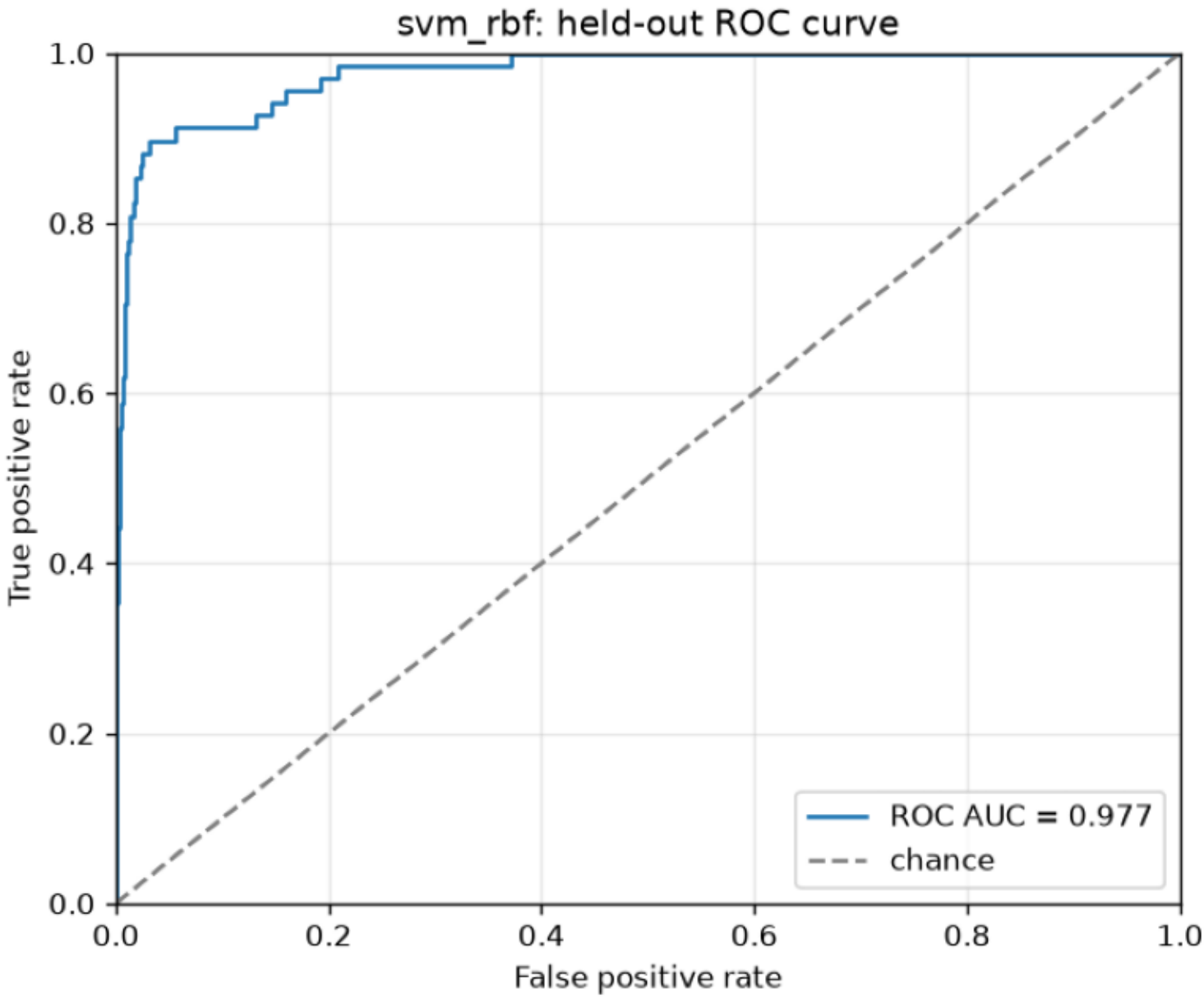
knn_roc_curve.png



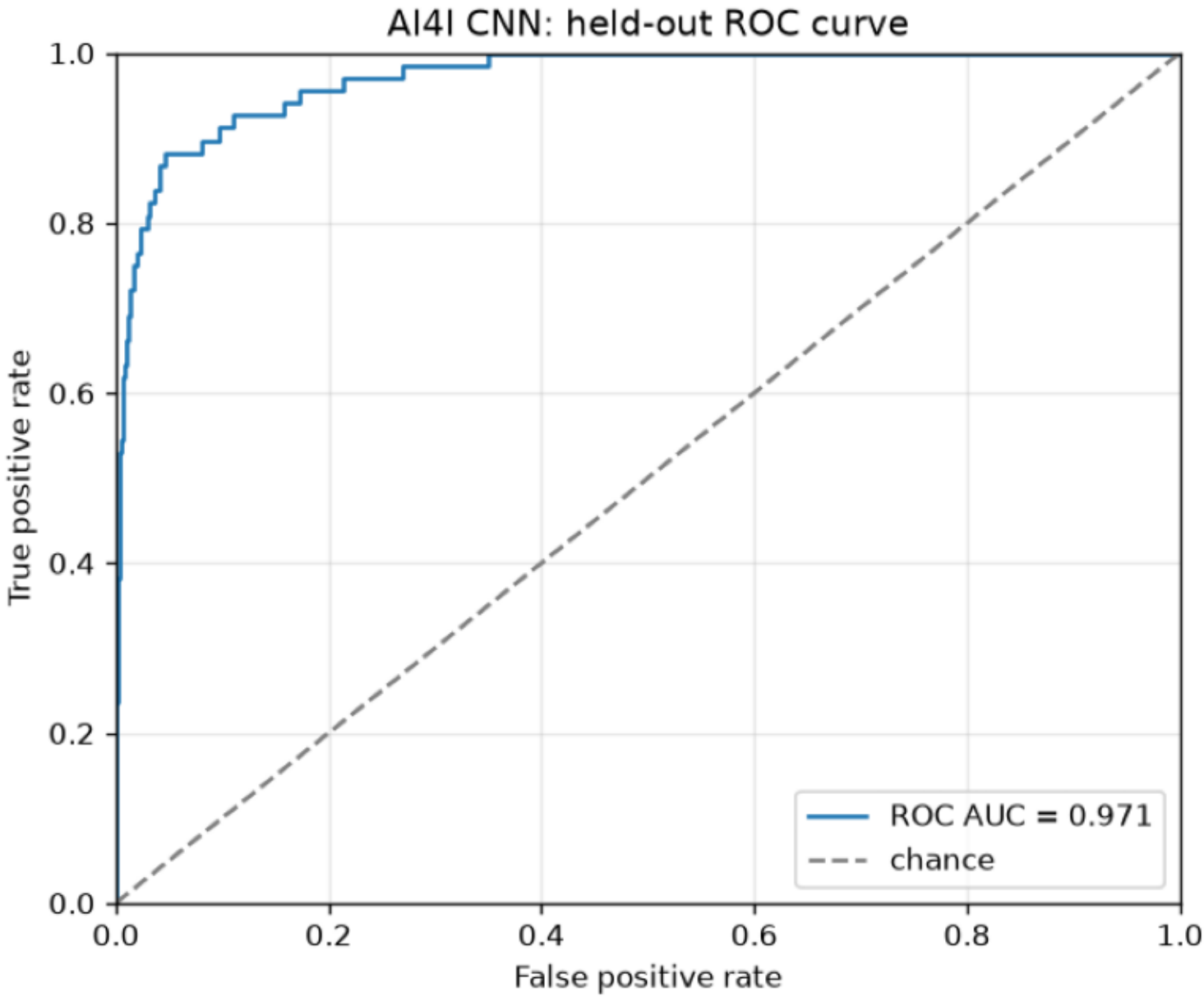
decision_tree_roc_curve.png



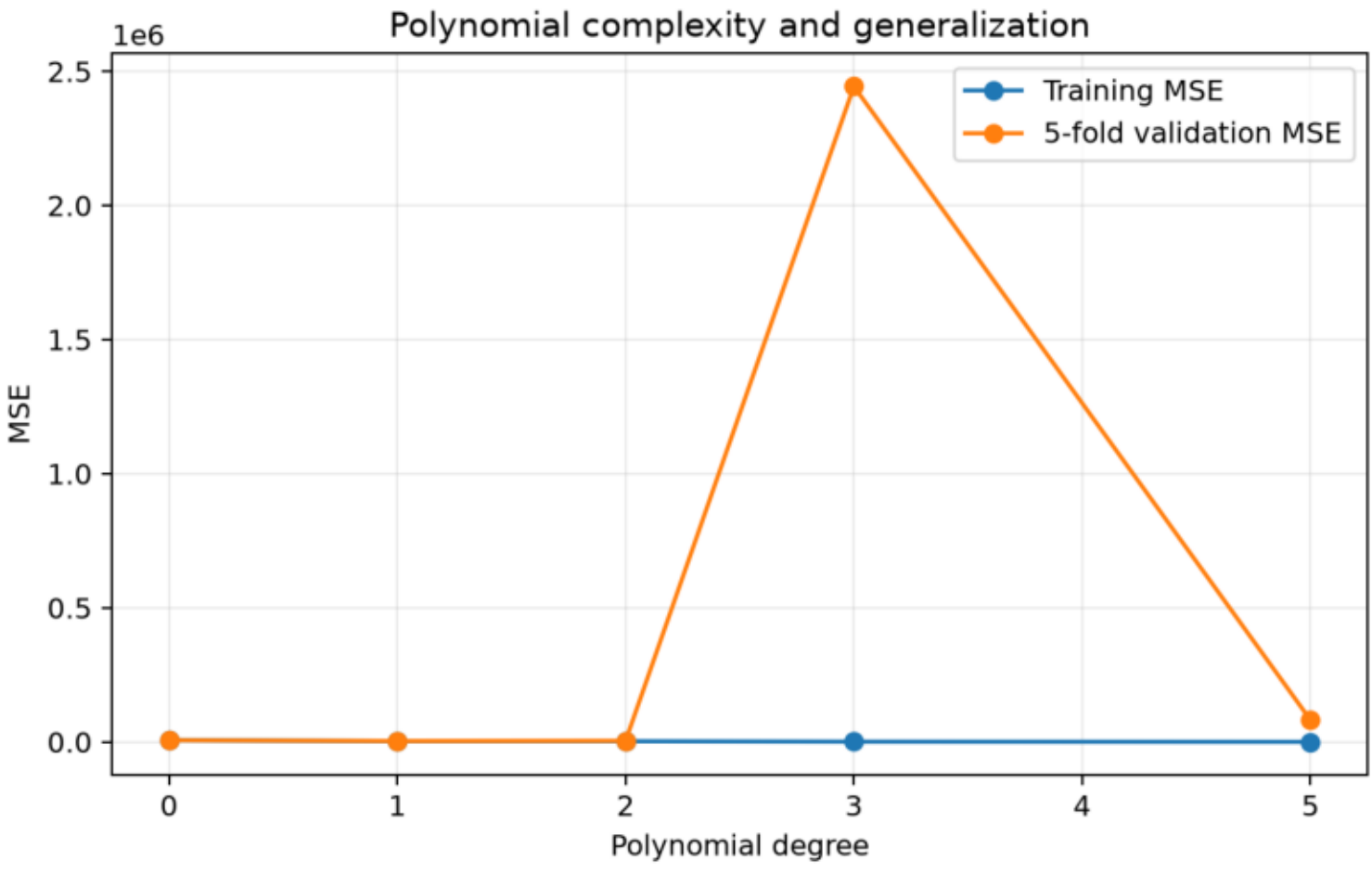
svm_rbf_roc_curve.png



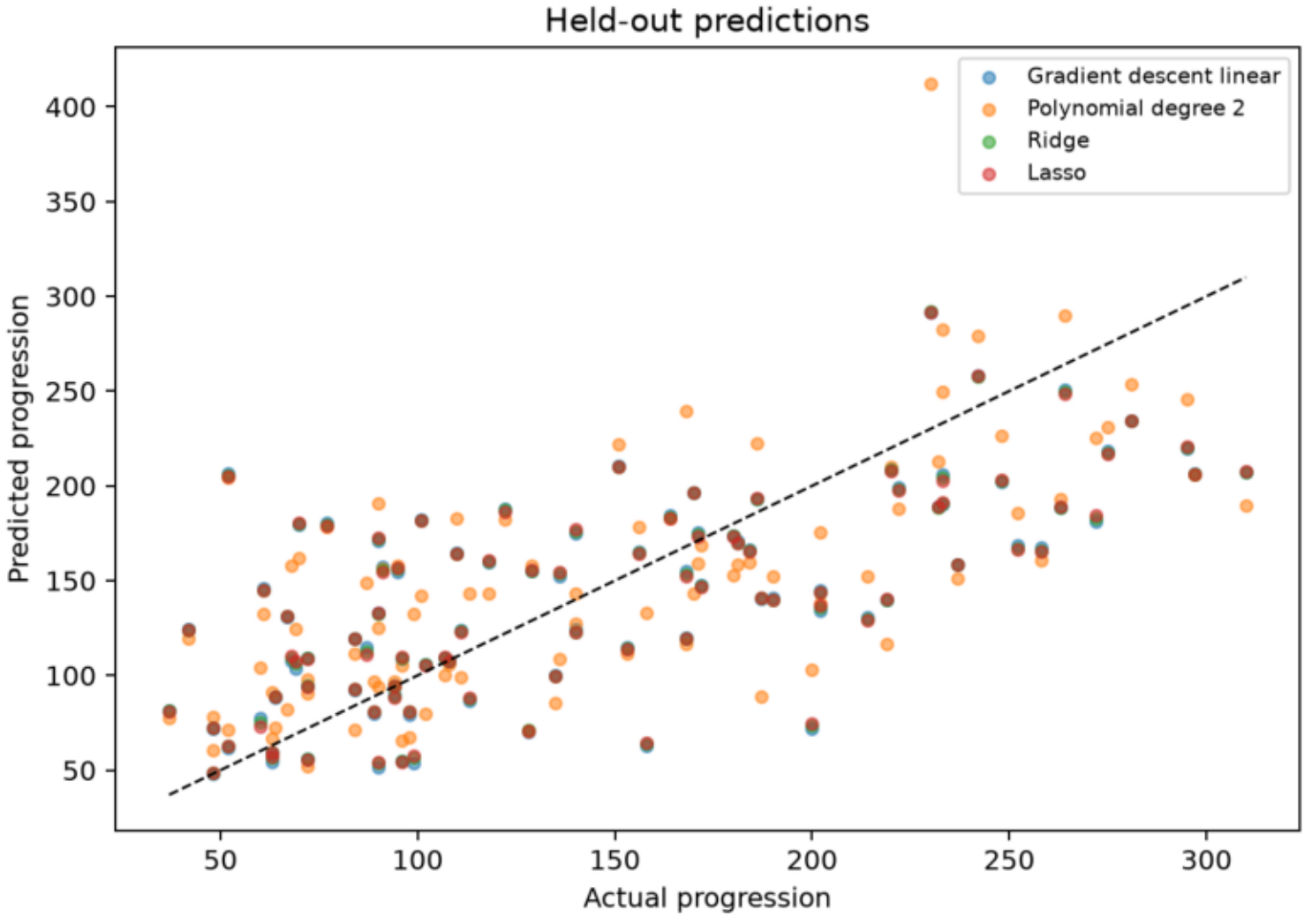
cnn_roc_curve.png



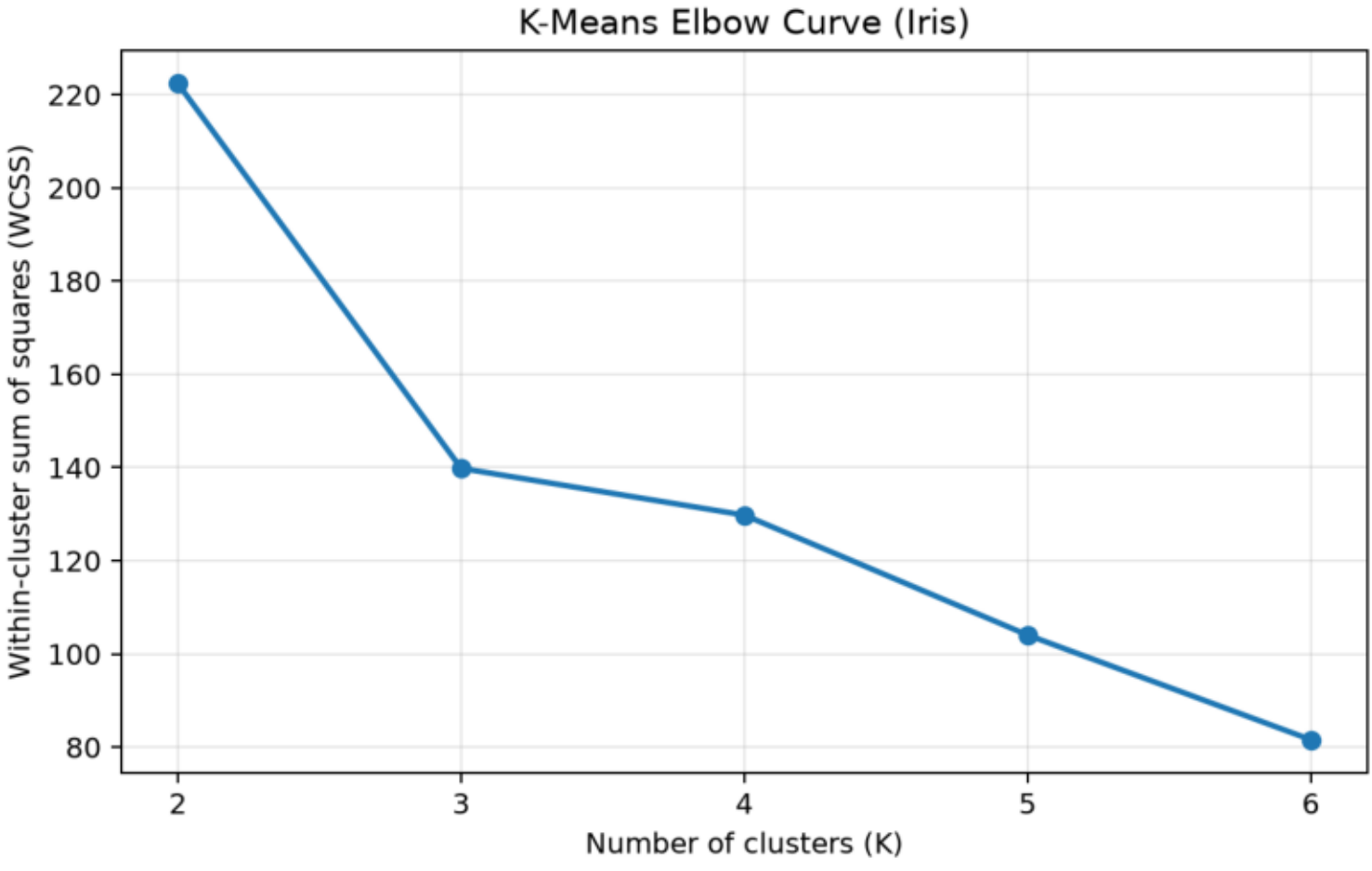
complexity_curve.png



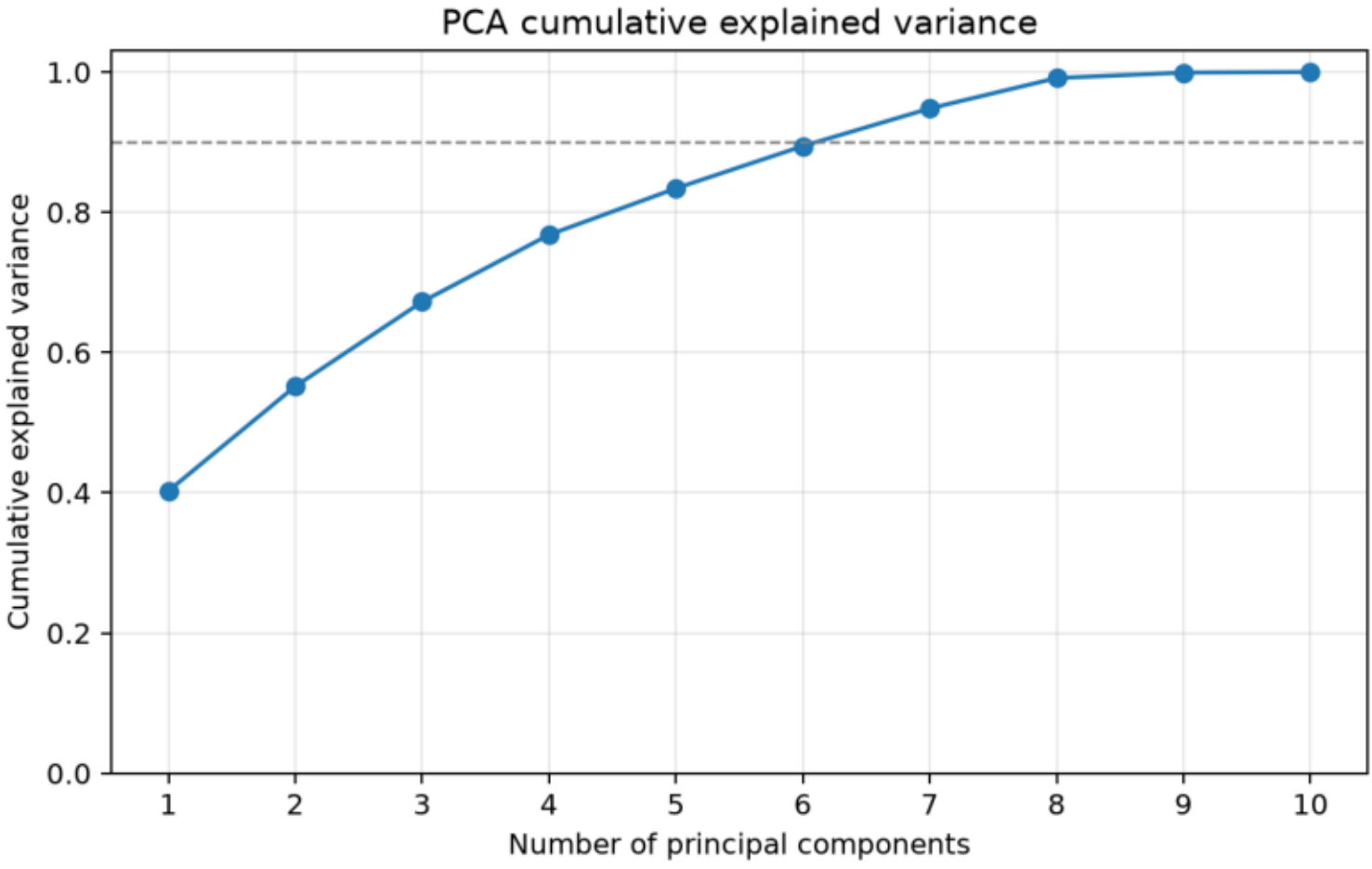
actual_vs_predicted.png



elbow_curve.png



cumulative_explained_variance.png



pca_2d_visualization.png

